LABORATORIO PRÁCTICO

Procesos e Hilos

Concurrencia, Paralelismo y Planificación de CPU

PROPÓSITO DEL LABORATORIO

Construir, ejecutar y analizar programas con procesos e hilos, relacionando el comportamiento observado con el scheduler, los núcleos físicos, los procesadores lógicos, la concurrencia, el paralelismo y el tiempo de ejecución.

Datos del laboratorio	Información
Asignatura	Sistemas Operativos
Modalidad	Laboratorio experimental en grupos
Lenguajes principales	C y/o C++ y/o Python
Sistemas operativos	Windows y/o Linux
Producto final	Código + evidencias + tablas + análisis técnico + conclusiones
Enfoque	No solo ejecutar: observar, medir, explicar y comparar

1. Presentación del laboratorio

Este laboratorio busca que el estudiante pase de la definición teórica de proceso e hilo a la observación experimental de cómo el sistema operativo administra la ejecución. Se trabajará con cargas de CPU, diferentes cantidades de hilos y procesos, herramientas de monitoreo y mediciones de tiempo.

La pregunta central es: ¿qué ocurre realmente cuando un programa crea varios hilos o varios procesos y el sistema operativo debe ejecutarlos sobre los procesadores lógicos disponibles?

IDEA CENTRAL

El sistema operativo planifica hilos sobre procesadores lógicos. Los procesadores lógicos pertenecen a núcleos físicos y estos forman parte de una CPU física instalada en un socket.

Se debe:

- Plantear el **problema**
- Solución del problema: Identificar **los procesos** requeridos para la solución del problema.
- Definir el número de hilos utilizados en cada proceso. Ejemplo: proceso 1, 5 hilos; proceso 2, 3 hilos; proceso 3, 7 hilos; etc.

2. Objetivos

- Comprender experimentalmente la diferencia entre proceso e hilo.
- Crear y ejecutar hilos en C y analizar su comportamiento.
- Observar la relación entre hilos, scheduler, procesadores lógicos y núcleos físicos.

- Identificar condiciones de concurrencia y paralelismo.
- Medir tiempos de ejecución con diferentes cantidades de hilos y procesos.
- Comparar el comportamiento observado en Windows y Linux.
- Interpretar técnicamente las evidencias obtenidas mediante herramientas del sistema operativo.

3. Resultados de aprendizaje

- Explica la jerarquía computador → socket → CPU → núcleo → procesador lógico → hilo.
- Distingue concurrencia de paralelismo mediante evidencia experimental.
- Relaciona número de hilos, capacidad de hardware, scheduler y tiempo de ejecución.
- Interpreta utilización de CPU y distribución de ejecución en Windows/Linux.
- Justifica resultados experimentales sin limitarse a presentar capturas de pantalla.

4. Marco conceptual mínimo

Concepto	Explicación breve
Proceso	Programa en ejecución con su propio espacio de direcciones y recursos. Tiene un identificador de proceso (PID).
Hilo	Unidad de ejecución dentro de un proceso. Los hilos del mismo proceso comparten memoria y muchos recursos, pero mantienen su propio estado de ejecución y pila.
CPU física	Procesador completo instalado en un socket. Puede contener múltiples núcleos.
Socket	Conector físico de la placa madre donde se instala una CPU física.
Núcleo físico	Unidad real de procesamiento dentro de una CPU; contiene recursos de ejecución.
Procesador lógico	Unidad de hardware visible para el sistema operativo sobre la cual puede planificar hilos. Con SMT puede haber varios por núcleo.
Scheduler	Componente del sistema operativo que decide qué hilo listo se ejecuta sobre qué procesador lógico.
Concurrencia	Varios trabajos avanzan de forma intercalada; no exige ejecución física simultánea.
Paralelismo	Dos o más hilos pueden ejecutar físicamente al mismo tiempo en diferentes procesadores lógicos.

FRASE PARA RECORDAR

El hilo es software; el procesador lógico es una unidad de hardware visible para el sistema operativo; el núcleo físico contiene los recursos que realizan físicamente la ejecución.

5. Estados simplificados de un hilo

```
NEW → READY → RUNNING → WAITING/BLOCKED → READY → RUNNING → TERMINATED
                                              ↑
                                   scheduler selecciona
                                              ↓
                                    procesador lógico
```

La llamada join no ejecuta el hilo: hace que el hilo que llama espere a que otro hilo termine. En POSIX, pthread_create crea el hilo y pthread_join espera su terminación.

6. Requisitos y preparación

6.1 Hardware y software

- Computador con Windows 10/11 o una distribución Linux actual.
- Compilador de C (GCC en Linux; MinGW-w64, MSYS2 o Visual Studio/Build Tools en Windows).
- Acceso a una herramienta de monitoreo de CPU.

- Permisos para compilar y ejecutar programas.

6.2 Recomendación de seguridad

Las cargas CPU-bound de este laboratorio pueden llevar uno o varios núcleos a alta utilización durante varios segundos o minutos. Utilice cargas de duración controlada, guarde los resultados y detenga la ejecución si el equipo presenta comportamiento anormal.

7. Experimento 1 — Identificación del hardware de ejecución

Antes de crear hilos, identifique cuántas CPUs físicas, núcleos y procesadores lógicos presenta el sistema operativo. Esto permite interpretar correctamente los experimentos posteriores.

7.1 Windows

Abra PowerShell y ejecute:

```
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores, NumberOfLogicalProcessors
```

La clase Win32_Processor expone NumberOfCores y NumberOfLogicalProcessors por instancia de procesador. En un sistema multiprocesador existe una instancia por procesador físico.

También puede consultar el total del sistema con:

```
Get-CimInstance Win32_ComputerSystem | Select-Object NumberOfProcessors, NumberOfLogicalProcessors
```

Microsoft documenta NumberOfProcessors como el número de procesadores físicos habilitados en el sistema y NumberOfLogicalProcessors como el número de procesadores lógicos.

7.2 Linux

Ejecute:

```
lscpu
lscpu -e
nproc
```

lscpu informa, entre otros datos, número de CPUs, hilos, núcleos, sockets y nodos NUMA; lscpu -e permite observar la topología por CPU lógica.

Dato a identificar	Valor de mi equipo	Interpretación
Socket(s)		
Core(s) per socket		
Thread(s) per core		
CPU(s) / procesadores lógicos		
Modelo de CPU		
NUMA node(s)		

Evidencia: capture la salida de los comandos y escriba debajo una interpretación de al menos 4–6 líneas.

Las observaciones del scheduler y la utilización de CPU se pueden experimentar de las siguientes formas:

Sistema Operativo	Lenguaje de Programación	Comandos y Herramientas del Sistema Operativo	Arquitectura de Hardware

Windows	C++	Python	Otro	Windows	Datos a identificar de la plataforma de ejecución y prueba del experimento
Linux	C++	Python	Otro	Linux	

8. Experimento 2 — Proceso frente a hilo

Construya primero una versión secuencial de una tarea CPU-bound. Luego cree variantes que ejecuten la misma carga mediante hilos y procesos. La comparación debe centrarse en tiempo, utilización de CPU y comportamiento del sistema.

8.1 Carga CPU-bound

Ejemplos: cálculo matemático repetitivo, búsqueda, simulación, compresión o procesamiento de datos. Para este laboratorio se recomienda una función determinista que realice operaciones suficientes para que el tiempo sea medible.

```
volatile unsigned long long acumulado = 0;

void trabajo(unsigned long long iteraciones) {
for (unsigned long long i = 0; i < iteraciones; ++i) {
acumulado += (i % 97) * (i % 53);
acumulado ^= (i << 3);
}
}
```

La variable volatile ayuda a evitar que una optimización agresiva elimine completamente el trabajo, pero no convierte el programa en un benchmark científico. Los tiempos deben interpretarse como resultados experimentales del equipo usado.

8.2 Procesos en Linux con fork()

En Linux, fork crea un proceso hijo. El proceso padre y el hijo continúan desde el punto de la llamada con espacios de memoria separados (con las particularidades de copia bajo demanda del sistema).

```
pid_t pid = fork();

if (pid < 0) {
perror("fork");
exit(EXIT_FAILURE);
}
else if (pid == 0) {
// Código del proceso hijo
trabajo(ITERACIONES);
_exit(0);
}
else {
// Código del proceso padre
wait(NULL);
}
```

Esta parte puede implementarse como experimento específico de Linux. En Windows, la creación de procesos se realiza mediante APIs distintas, por ejemplo CreateProcess; no debe asumirse que fork existe de forma nativa.

9. Experimento 3 — Creación de hilos en C

9.1 Linux/POSIX: pthread_create y pthread_join

En POSIX, pthread_create crea un nuevo hilo dentro del proceso y comienza su rutina de ejecución. pthread_join hace que el hilo llamador espere a que el hilo objetivo termine.

```
#include <pthread.h>
 #include <stdio.h>
 #include <stdlib.h>

  #define N_HILOS 4

void *trabajo(void *arg) {
     int id = *(int *)arg;
printf("Hilo %d ejecutando...\n", id);

 // Aquí va la carga CPU-bound
  // trabajo_cpu(ITERACIONES);

         return NULL;
             }

    int main(void) {
   pthread_t hilos[N_HILOS];
       int ids[N_HILOS];

 for (int i = 0; i < N_HILOS; ++i) {
              ids[i] = i;
pthread_create(&hilos[i], NULL, trabajo, &ids[i]);
                  }

 for (int i = 0; i < N_HILOS; ++i) {
      pthread_join(hilos[i], NULL);
                  }

printf("Todos los hilos terminaron.\n");
              return 0;
                }
```

```
gcc -O0 -pthread hilos.c -o hilos
./hilos
```

OBSERVACIÓN IMPORTANTE

Crear un hilo no significa que inmediatamente se ejecute en un núcleo concreto. Después de pthread_create, el scheduler determina cuándo obtiene tiempo de CPU. El comportamiento exacto de la siguiente instrucción puede variar entre ejecuciones.

9.2 Windows

En Windows puede realizarse el mismo experimento mediante la API nativa CreateThread. Si se utiliza un entorno con pthreads compatible, también puede emplearse la interfaz POSIX; en el informe debe indicarse claramente cuál API y compilador se utilizaron.

```
#include <windows.h>
 #include <stdio.h>

DWORD WINAPI trabajo(LPVOID arg) {
     int id = *(int*)arg;
printf("Hilo %d ejecutando...\n", id);

  // Carga CPU-bound

       return 0;
         }

  int main(void) {
    const int N = 4;
    HANDLE hilos[N];
      int ids[N];
```

```
                for (int i = 0; i < N; ++i) {
                    ids[i] = i;
    hilos[i] = CreateThread(NULL, 0, trabajo, &ids[i], 0, NULL);
                }

    WaitForMultipleObjects(N, hilos, TRUE, INFINITE);

                for (int i = 0; i < N; ++i)
                    CloseHandle(hilos[i]);

                return 0;
                }
```

10. Experimento 4 — Concurrencia y paralelismo con 1, 2, 4 y 8 hilos

Ejecute la misma carga CPU-bound cambiando únicamente el número de hilos. Si el equipo dispone de más de 8 procesadores lógicos, puede ampliar a 16 o 32. Si dispone de menos, use los valores disponibles y explique la limitación.

Ejecución	N.º de hilos	Tiempo (s)	CPU total	Observación sobre LP/núcleos
Secuencial	1			
A	2			
B	4			
C	8			
D	16 (opcional)			

10.1 Prueba de concurrencia

Cuando se restringe la ejecución a un solo procesador lógico, varios hilos pueden avanzar de manera intercalada: existe concurrencia, pero no paralelismo físico simultáneo.

Esta prueba es especialmente útil si el entorno permite establecer afinidad de CPU. Si no puede fijarse la afinidad, describa la limitación y utilice la observación temporal y del scheduler como evidencia complementaria.

10.2 Prueba de paralelismo

Con varios procesadores lógicos disponibles, dos o más hilos CPU-bound independientes pueden ejecutarse físicamente al mismo tiempo. Esto es paralelismo. No significa que cada hilo quede permanentemente asignado a un núcleo: el scheduler puede migrarlos.

CONDICIÓN CLAVE

Un hilo no se ejecuta simultáneamente en varios núcleos. Puede migrar entre procesadores lógicos a lo largo de su vida.

11. Experimento 5 — Observación del scheduler y la utilización de CPU

11.1 Windows

- Administrador de tareas → Rendimiento → CPU: observe núcleos, procesadores lógicos y utilización.
- Administrador de tareas → Detalles: localice el proceso del laboratorio.
- Monitor de recursos: observe actividad de CPU por proceso/hilo cuando esté disponible.
- PowerShell: utilice Get-CimInstance para registrar la topología del procesador.

11.2 Linux

- top: observe procesos y utilización.
- htop: observe procesos/hilos y utilización por CPU lógica.
- ps -eLf: observe información de procesos e hilos cuando esté disponible.
- pidstat -t -p PID 1: observe actividad por hilo de un proceso.
- lscpu -e: observe la topología de procesadores lógicos.

```
# Ejemplo Linux
./hilos &
PID=$!
ps -eLf | grep $PID
pidstat -t -p $PID 1
```

Las herramientas pueden mostrar CPU lógica, tiempo de CPU, porcentaje de utilización y otros datos. La salida exacta depende de la versión del sistema y del entorno.

12. Experimento 6 — Procesos vs. hilos

Repita una carga CPU-bound comparable con N hilos y con N procesos. El objetivo no es demostrar que una técnica siempre sea más rápida, sino analizar el costo de creación, planificación, memoria, comunicación y sincronización.

Característica	Hilos	Procesos
Memoria	Comparten el espacio de direcciones del proceso.	Cada proceso posee su propio espacio de direcciones.
Comunicación	Puede usar variables compartidas, mutex, condiciones, etc.	Requiere mecanismos de IPC u otros medios.
Creación	Generalmente menor costo que un proceso independiente.	Generalmente mayor costo.
Aislamiento	Menor: un error puede afectar al proceso completo.	Mayor aislamiento entre procesos.
Paralelismo	Sí, con varios procesadores lógicos disponibles.	Sí, con varios procesadores lógicos disponibles.
Uso didáctico	Concurrencia, sincronización y paralelismo.	Aislamiento, recursos y paralelismo a nivel de procesos.

13. Experimento 7 — Medición del tiempo

No compare una sola ejecución. Realice al menos 3 repeticiones por configuración y calcule promedio y, si es posible, desviación o rango. Cierre aplicaciones innecesarias y mantenga constante la carga.

Configuración	R1 (s)	R2 (s)	R3 (s)	Promedio (s)	CPU/LP observado
1 hilo					
2 hilos					
4 hilos					
8 hilos					
16 hilos (opcional)					

Una reducción de tiempo no tiene por qué ser proporcional al número de hilos. Existen costos de creación, planificación, sincronización, competencia por cachés y memoria, y otros procesos del sistema.

14. Interpretación obligatoria de resultados

REGLA DE EVALUACIÓN

No se calificará únicamente la obtención de resultados. Se calificará principalmente la capacidad del estudiante para interpretar técnicamente lo observado y relacionarlo con los conceptos estudiados.

Cada evidencia debe responder las siguientes tres preguntas:

1. ¿Qué se observa?
2. ¿Por qué ocurre?
3. ¿Qué demuestra respecto al experimento?

Ejemplo de una interpretación insuficiente: “Con 8 hilos la CPU marcó 100 %”.

Ejemplo de una interpretación adecuada: “Al aumentar de 4 a 8 hilos, la utilización total de CPU aumentó porque existen más unidades de trabajo listas para ser planificadas. Sin embargo, el tiempo no disminuyó en la misma proporción debido a la competencia por recursos compartidos y al costo de planificación. La observación es coherente con un sistema que dispone de varios procesadores lógicos y permite paralelismo de los hilos CPU-bound.”

15. Preguntas de análisis

4. ¿Cuántas CPUs físicas, sockets, núcleos físicos y procesadores lógicos tiene su equipo?
5. ¿La cantidad de procesadores lógicos coincide con la cantidad de núcleos? ¿Por qué?
6. ¿Qué evidencia permite afirmar que hubo concurrencia?
7. ¿Qué evidencia permite afirmar que hubo paralelismo?
8. ¿Qué ocurre cuando el número de hilos supera el número de procesadores lógicos?
9. ¿El tiempo de ejecución disminuye linealmente al duplicar los hilos? Explique.
10. ¿Puede un hilo migrar entre procesadores lógicos? ¿Cómo podría observarlo?
11. ¿Qué diferencia observó entre ejecutar varios hilos y varios procesos?
12. ¿Qué limitaciones tuvo el experimento?
13. ¿Qué diferencias encontró entre Windows y Linux?

16. Comparación Windows vs. Linux

Aspecto	Windows	Linux
Identificación de CPU	Administrador de tareas / PowerShell	lscpu / lscpu -e
Procesos	Administrador de tareas / PowerShell	ps / top / htop
Hilos	Detalles/monitoreo; APIs Win32 o entorno POSIX	pthread + ps/htop/pidstat
Creación de proceso en C	CreateProcess (API nativa)	fork / exec / wait
Creación de hilo en C	CreateThread (API nativa)	pthread_create
Esperar hilo	WaitFor... / APIs de sincronización	pthread_join
Análisis	Comparar topología, CPU y tiempos	Comparar topología, CPU, hilos y tiempos

17. Evidencias y productos a entregar

- Código fuente comentado.
- Captura o registro de la identificación de hardware.
- Capturas o registros de las ejecuciones.
- Tabla de tiempos para cada configuración.
- Gráfica de tiempo de ejecución vs. número de hilos.
- Gráfica de utilización de CPU vs. número de hilos, si la herramienta lo permite.
- Tabla comparativa Windows/Linux.
- Respuestas a las preguntas de análisis.
- Conclusiones técnicas.

18. Estructura sugerida del informe

Sección	Contenido
1. Introducción	Problema, propósito y contexto.
2. Marco conceptual	Proceso, hilo, CPU, núcleo, procesador lógico, scheduler, concurrencia y paralelismo.
3. Caracterización del equipo	Sistema operativo, CPU, sockets, núcleos y procesadores lógicos.
4. Implementación	Código y explicación de procesos/hilos.
5. Experimentos	Procedimiento, configuraciones y herramientas.
6. Resultados	Tablas, gráficas y evidencias.
7. Análisis	Interpretación técnica de los resultados.
8. Comparación Windows/Linux	Similitudes, diferencias y limitaciones.
9. Conclusiones	Hallazgos principales y relación con los objetivos.
10. Referencias	Fuentes consultadas.

19. Rúbrica de evaluación sugerida

Criterio	Peso	Indicador
Fundamentos	15 %	Explica correctamente proceso, hilo, CPU, núcleo, LP y scheduler.
Implementación	20 %	Código funcional, claro y comentado.
Diseño experimental	15 %	Variables controladas, repeticiones y procedimiento reproducible.
Medición	15 %	Tiempos, tablas y gráficas correctamente construidos.
Interpretación	25 %	Relaciona evidencia con concurrencia, paralelismo, scheduler y hardware.
Presentación	10 %	Informe claro, ordenado y con conclusiones sustentadas.

20. Conclusión esperada

Al finalizar, el estudiante debe poder explicar experimentalmente la cadena:

```
PROGRAMA → PROCESO → HILOS → SCHEDULER → PROCESADORES LÓGICOS → NÚCLEOS FÍSICOS → EJECUCIÓN
```

La conclusión no debe limitarse a indicar qué programa fue más rápido. Debe explicar qué ocurrió, por qué ocurrió y qué relación tiene con la arquitectura de hardware y la planificación del sistema operativo.

MENSAJE FINAL

El sistema operativo controla y planifica; los hilos utilizan los procesadores lógicos; el hardware ejecuta las instrucciones.

21. Referencias técnicas para la práctica

Las siguientes referencias se utilizaron para verificar la sintaxis y el significado de los comandos y APIs indicados en esta guía:

- Linux man-pages: lscpu — información de arquitectura de CPU, sockets, núcleos, hilos y NUMA.
- Linux man-pages: pthread_create — creación de hilos POSIX.
- Linux man-pages: pthread_join — espera de terminación de un hilo.
- Microsoft Learn: Win32_Processor — NumberOfCores y NumberOfLogicalProcessors.
- Microsoft Learn: Win32_ComputerSystem — NumberOfProcessors y NumberOfLogicalProcessors.
- Microsoft Learn: Get-CimInstance — consulta de información del sistema mediante CIM.